

ui.date 全面详解

`ui.date` 是 NiceGUI 中基于 Quasar 的 `QDate` 组件封装的日期选择器核心组件，支持单日期、日期范围、多日期选择等多种模式，提供日期格式自定义、日期过滤、数据绑定等丰富功能，是构建日期交互界面的基础组件。3.3.0 版本后新增的 `ui.date_input` 是其封装后的便捷输入组件，而 `ui.date` 更适合需要自定义触发方式、布局或深度配置的场景（如弹窗式日期选择、复杂表单内嵌等）。

一、核心初始化参数

初始化 `ui.date` 时需关注 3 个核心参数，覆盖基础日期选择需求，同时支持通过 `props` 扩展高级功能：

参数名	说明
<code>value</code>	初始日期值，格式随选择模式变化：- 单日期模式：字符串（如 <code>2023-01-01</code> ）- 范围模式：字典（ <code>{'from': '2023-01-01', 'to': '2023-01-05'}</code> ）- 多日期模式：列表（如 <code>['2023-01-01', '2023-01-02']</code> ）- 多范围模式：列表（如 <code>[{'from': '2023-01-01', 'to': '2023-01-05'}, '2023-01-07']</code> ）
<code>mask</code>	日期字符串格式（默认 <code>'YYYY-MM-DD'</code> ），支持 Quasar 日期格式占位符（如 <code>'YYYY/MM/DD'</code> 、 <code>'MM-DD-YYYY'</code> 等）
<code>on_change</code>	日期值变更时触发的回调函数，事件对象通过 <code>e.value</code> 获取当前选中日期 / 范围 / 多日期数据

基础使用示例

1. 单日期选择（默认模式）

```
from nicegui import ui

result = ui.label('选中日期：')

# 初始化单日期选择器，初始值 2023-01-01
ui.date(
    value='2023-01-01',
    mask='YYYY-MM-DD',
    on_change=lambda e: result.set_text(f'选中日期: {e.value}')
)

ui.run()
```

2. 日期范围选择 (通过 `props('range')` 启用)

```
from nicegui import ui

result = ui.label('选中范围: ')

# 初始化日期范围选择器, 初始值为字典格式
range_picker = ui.date(
    value={'from': '2023-01-01', 'to': '2023-01-05'},
    on_change=lambda e: result.set_text(f'选中范围: {e.value["from"]} 至 {e.value["to"]}'),
).props('range') # 启用范围选择模式

ui.run()
```

3. 多日期选择 (通过 `props('multiple')` 启用)

```
from nicegui import ui

result = ui.label('选中日期: ')

# 初始化多日期选择器, 初始值为列表格式
multi_picker = ui.date(
    value=['2023-01-01', '2023-01-02', '2023-01-03'],
    on_change=lambda e: result.set_text(f'选中日期: {"", " ".join(e.value)}'),
).props('multiple') # 启用多日期选择模式

ui.run()
```

4. 多范围选择 (同时启用 `range` 和 `multiple`)

```
from nicegui import ui

result = ui.label('选中范围: ')

# 初始化多范围选择器, 初始值为混合列表格式
multi_range_picker = ui.date(
    value=[{'from': '2023-01-01', 'to': '2023-01-05'}, '2023-01-07'],
    on_change=lambda e: result.set_text(f'选中范围: {e.value}'),
).props('multiple range') # 同时启用多选择和范围模式

ui.run()
```

二、核心特性: 模式扩展与高级配置 (通过 `props` 属性)

`ui.date` 的强大之处在于通过 `props` 支持丰富的模式扩展和行为配置, 基于 Quasar `QDate` 的原生属性, 可实现多样化需求:

常用 props 配置

props 参数	说明
range	启用日期范围选择模式（value 为字典或多范围列表）
multiple	启用多日期选择模式（value 为列表）
default-year-month	设置默认显示的年月（格式 YYYY/MM，如 '2023/06'）
no-today	隐藏“今天”快捷按钮
no-clear	隐藏“清除”快捷按钮
readonly	只读模式，禁止修改日期
disabled	禁用组件，禁止用户交互
:options	日期过滤规则（Vue 表达式，如 date => date <= '2023/01/15'）

高级配置示例

1. 日期过滤（仅允许选择 2023-01-15 及之前的日期）

```
from nicegui import ui

# 配置日期过滤规则：日期 <= 2023/01/15（注意 props 中日期格式为 YYYY/MM/DD）
ui.date().props('default-year-month=2023/01 :options="date => date <= \'2023/01/15\'"')

ui.run()
```

2. 自定义默认年月与隐藏快捷按钮

```
from nicegui import ui

# 默认显示 2023 年 6 月，隐藏“今天”和“清除”按钮
ui.date(
    value='2023-06-10',
    on_change=lambda e: ui.notify(f'选中日期: {e.value}')
).props('default-year-month=2023/06 no-today no-clear')

ui.run()
```

三、组件核心属性

ui.date 继承 NiceGUI 基础元素的核心属性，支持动态状态控制、样式自定义、DOM 标识等能力，关键属性如下：

属性名	类型	说明
classes	Classes[Self]	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观（如宽度、边框、间距）

属性名	类型	说明
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定， <code>False</code> 时禁止用户交互，支持动态启用 / 禁用）
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，用于精准 DOM 操作）
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读属性，用于判断组件生命周期状态）
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读属性，用于调试或事件控制场景）
<code>parent_slot</code>	<code>Slot None</code>	组件的父插槽（可设置，用于复杂布局中的插槽嵌套）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（用于扩展模式、行为等，如 <code>range</code> 、 <code>multiple</code> ）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式（如 <code>width: 300px; margin: 10px 0</code> ）
<code>value</code>	<code>BindableProperty</code>	当前选中的日期 / 范围 / 多日期值（支持动态绑定，可通过 <code>set_value</code> 方法修改）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（支持动态绑定， <code>False</code> 时隐藏组件）

属性操作示例

```
from nicegui import ui

# 初始化单日期选择器
date_picker = ui.date(value='2023-01-01')

# 按钮控制组件状态
ui.button('禁用选择器', on_click=date_picker.disable)
ui.button('启用选择器', on_click=date_picker.enable)
ui.button('设置为 2023-02-01', on_click=lambda: date_picker.set_value('2023-02-01'))
ui.button('隐藏选择器', on_click=lambda: date_picker.set_visibility(False))

ui.run()
```

四、核心方法

`ui.date` 提供完善的方法用于组件状态控制、数据绑定、样式自定义等，按功能分类如下：

1. 状态控制方法

用于直接修改组件的启用状态、可见性、值等基础属性：

方法名	说明
<code>enable()</code>	启用组件（允许用户交互）
<code>disable()</code>	禁用组件（禁止用户交互，组件变灰）
<code>set_enabled(value: bool)</code>	动态设置启用状态（ <code>True</code> 启用， <code>False</code> 禁用）
<code>set_value(value: Any)</code>	设置日期值：- 单日期：字符串（如 <code>'2023-01-01'</code> ）- 范围：字典（如 <code>{'from': '2023-01-01', 'to': '2023-01-05'}</code> ）- 多日期：列表（如 <code>['2023-01-01', '2023-01-02']</code> ）
<code>set_visibility(visible: bool)</code>	动态设置组件可见性（ <code>True</code> 显示， <code>False</code> 隐藏）
<code>clear()</code>	清除所有子元素（极少用于日期选择器，适用于嵌套复杂内容场景）
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）

2. 数据绑定方法

支持将组件的 `enabled`、`value`、`visible` 等属性与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_enabled(...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_value(...)</code>	双向绑定组件日期值与目标对象属性（核心绑定方法，支持格式转换）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）日期值
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）日期值
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例（日期范围与输入框同步）

```
from nicegui import ui

# 创建输入框，用于显示/编辑日期范围字符串
date_input = ui.input('日期范围').classes('w-60')

# 日期范围选择器与输入框双向绑定，通过 forward/backward 转换格式
ui.date().props('range').bind_value(
```

```

date_input,
# 正向转换: 将日期选择器的字典值转为 "from - to" 字符串
forward=lambda x: f'{x["from"]} - {x["to"]}' if x else None,
# 反向转换: 将输入框的字符串转为字典值
backward=lambda x: {
    'from': x.split(' - ')[0],
    'to': x.split(' - ')[1],
} if ' - ' in (x or '') else None,
)

ui.run()

```

3. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能:

方法名	说明
<code>classes(add/remove/toggle/replace)</code>	新增 / 移除 / 切换 / 替换组件的 HTML 类 (如 <code>date_picker.classes('w-40 border-blue-500')</code>)
<code>style(add/remove/replace)</code>	新增 / 移除 / 替换组件的内联 CSS 样式 (如 <code>date_picker.style('font-size: 14px;')</code>)
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类 (需在实例化前调用, 如统一设置宽度)
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式 (需在实例化前调用)
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示 (鼠标悬浮时显示, 如 <code>tooltip('选择日期')</code>)
<code>add_resource(path)</code>	为组件添加资源文件 (如自定义 CSS/JS, 用于扩展样式或功能)
<code>mark(*markers)</code>	为组件添加标记 (用于测试查询或依赖管理)

4. 其他实用方法

方法名	说明
<code>ancestors(include_self)</code>	迭代组件的祖先元素 (<code>include_self=True</code> 包含自身)
<code>descendants(include_self)</code>	迭代组件的子元素 (<code>include_self=True</code> 包含自身)
<code>get_computed_prop(prop_name, timeout)</code>	获取计算属性 (需异步等待, 如获取组件实际渲染后的宽度)
<code>move(target_container, target_index, target_slot)</code>	移动组件到其他容器 (用于动态布局调整)
<code>on(type, handler, ...)</code>	订阅通用 DOM 事件 (如点击、鼠标悬浮等)

方法名	说明
<code>on_value_change(callback)</code>	绑定日期值变更事件（与初始化 <code>on_change</code> 参数等价）
<code>remove(element)</code>	移除子元素（极少使用）
<code>run_method(name, *args)</code>	运行客户端方法（如调用底层 Quasar 组件的原生方法）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改过滤规则后刷新）

五、高级场景：自定义触发方式（输入框 + 图标触发）

`ui.date` 本身是纯日期选择面板，可结合 `ui.input`、`ui.menu`、`ui.icon` 实现类似 `ui.date_input` 的“输入框 + 图标触发”交互，支持更灵活的布局自定义：

```
from nicegui import ui

# 创建输入框，用于显示日期
with ui.input('选择日期') as date_input:
    # 创建菜单，用于包裹日期选择器
    with ui.menu().props('no-parent-event') as menu:
        # 日期选择器与输入框双向绑定
        with ui.date().bind_value(date_input):
            # 添加关闭按钮（菜单默认无关闭按钮，手动添加提升体验）
            with ui.row().classes('justify-end mt-2'):
                ui.button('关闭', on_click=menu.close).props('flat')
        # 在输入框右侧添加图标，点击图标打开菜单
        with date_input.add_slot('append'):
            ui.icon('edit_calendar').on('click', menu.open).classes('cursor-pointer text-gray-500')

ui.run()
```

关键说明

- `ui.menu().props('no-parent-event')`：防止点击输入框时触发菜单打开，仅通过图标触发；
- `date_input.add_slot('append')`：在输入框右侧添加图标插槽；
- 日期选择器与输入框通过 `bind_value` 双向绑定，确保数据同步。

六、事件处理

1. 核心事件：日期值变更（`on_change/on_value_change`）

日期值变更事件是 `ui.date` 的核心事件，支持两种绑定方式（功能等价）：

```
# 方式1: 初始化时通过 on_change 参数绑定
ui.date(
    value='2023-01-01',
    on_change=lambda e: print(f'日期变更为: {e.value}')
)

# 方式2: 通过 on_value_change 方法绑定
date_picker = ui.date(value='2023-01-01')
date_picker.on_value_change(lambda e: print(f'日期变更为: {e.value}'))
```

2. 通用事件订阅 (on 方法)

通过 on() 方法订阅其他 DOM 事件（如点击、鼠标悬浮、聚焦等），支持客户端 JS 处理或服务端 Python 处理：

```
from nicegui import ui

date_picker = ui.date(value='2023-01-01')

# 订阅点击事件（服务端处理）
date_picker.on('click', lambda: print('日期选择器被点击'))

# 订阅鼠标悬浮事件（客户端 JS 处理）
date_picker.on(
    'mouseover',
    js_handler='(e) => console.log("鼠标悬浮在日期选择器上")'
)

ui.run()
```

七、版本兼容性说明

特性 / 属性	支持版本
html_id 属性	2.16.0+
toggle 参数 (default_classes)	2.7.0+
strict 参数 (绑定方法)	3.0.0+
同时指定 Python 和 JS 事件处理 (on 方法)	2.18.0+
ui.date_input 替代方案提示	3.3.0+

八、与 ui.date_input 的核心区别

特性	ui.date	ui.date_input
核心形态	纯日期选择面板（无输入框）	输入框 + 日期选择器（一体化组件）
触发方式	需手动结合菜单 / 图标触发	点击输入框自动弹出选择器

特性	ui.date	ui.date_input
适用场景	自定义布局、弹窗式选择、深度配置	表单内嵌、快速实现日期输入功能
初始化复杂度	较高（需手动组合组件）	较低（开箱即用）
核心优势	灵活性强、支持深度定制	便捷性高、集成度高

简单来说：快速实现表单日期输入用 `ui.date_input`；需要自定义触发方式、布局或深度配置日期选择规则用 `ui.date`。

总结

`ui.date` 是 NiceGUI 中功能最基础也最灵活的日期选择组件，核心优势在于：

- 1. 支持单日期、范围、多日期、多范围四种选择模式，覆盖复杂日期选择需求；
- 2. 基于 Quasar QDate 组件，提供丰富的 `props` 配置（日期过滤、快捷按钮控制等）；
- 3. 完善的数据绑定能力，支持跨组件数据同步与格式转换；
- 4. 样式自定义能力强，可通过类、内联样式适配不同界面风格；
- 5. 可与 `ui.input`、`ui.menu` 等组件组合，实现自定义交互逻辑。

适用于弹窗式日期选择、复杂表单内嵌、自定义日期输入组件等场景，是构建日期交互界面的核心基础组件。